Pimple Removal Tool: Acne Detection and Shared-Mask Image Inpainting

Yogdeep Singh Rajawat
github.com/blackleg-sanji007

Abstract

Acne removal from photographs requires two distinct decisions: which pixels contain lesions and how to reconstruct plausible skin after those pixels are removed. A restoration model applied to the whole face can change unaffected features, while a detector alone cannot produce a cleaned image. This project combines a YOLOv8s acne detector trained for 60 epochs on 1,457 labeled Acne04 images with two inpainting paths that receive the same predicted, dilated lesion mask: a pretrained DeepFillv2 generator and OpenCV's Telea method. Its system-level contribution is a reproducible detector-to-mask pipeline and an interactive interface with original/DeepFillv2 and OpenCV/DeepFillv2 comparison sliders. The best validation checkpoint, obtained at epoch 57, reached 40.37% precision, 42.20% recall, 32.53% mAP@50, and 8.00% mAP@50:95. On a separate 146-image test split it reached 42.39% precision, 42.74% recall, 32.79% mAP@50, and 8.51% mAP@50:95. A held-out visual example shows the predicted mask and both restorations. DeepFillv2 appears more natural on that example, whereas the Telea result contains visible patches. Acne04 has lesion boxes but no paired clean face targets, so the study does not claim a numerical restoration quality advantage.

1. Introduction

Portrait retouching is a common image-editing task, but manual removal of many small acne lesions is slow. Automating the task is difficult because lesions differ in size, color, and contrast; many are close to eyes, lips, hair, or textured skin. A system must identify the intended edit region and reconstruct it without unnecessarily altering surrounding facial features. For a user, a convincing result also needs to be inspectable against the original image.

The project addresses these requirements with a two-stage method. YOLOv8s [7] predicts acne bounding boxes. These boxes become a binary mask, which is expanded slightly to include lesion boundaries. The same mask is sent to the gated-convolution DeepFillv2 inpainting implementation [8, 9] and to the fast-marching-based Telea implementation in OpenCV [4, 6]. A Gradio interface returns both images and draggable comparisons. The pipeline was adapted from an earlier YOLOv8–DeepFillv2 course project [2]; this implementation adds an independently trained 60-epoch checkpoint, a shared-mask classical baseline, and an interactive comparison interface.

The novelty here is the evaluation-oriented integration. By holding the detector output and mask fixed for both restoration paths, the visual comparison focuses on the inpainting methods rather than different lesion selections. The interface makes changed regions and possible artifacts visible to a user. The detector architecture, DeepFillv2 generator, and Telea algorithm are established methods. This report documents what was trained, what was reused, and what the available data can actually measure.

The contributions are: (1) an Acne04 training workflow with a fixed train/validation/test split and a committed best detector checkpoint; (2) a detector-to-mask-to-inpainting application with two methods using the same mask; and (3) measured detection results, a held-out visual comparison, and complete project-authored source code in the appendix.

2. Proposed Method

Figure 1 summarizes the training and inference paths. Training yields a detector checkpoint once. A new uploaded face then runs through detection, mask construction, two restoration branches, and a comparison display without retraining.

2.1. Dataset preparation and detector training

The Acne04 YOLOv8-format dataset [1] contains 1,457 image/label pairs used in this run. Each label file stores pimple boxes in YOLO format. The training script sorts the paired files and applies two train_test_split calls with random state 42. This produces 1,019 training, 292 validation, and 146 test images, approximately

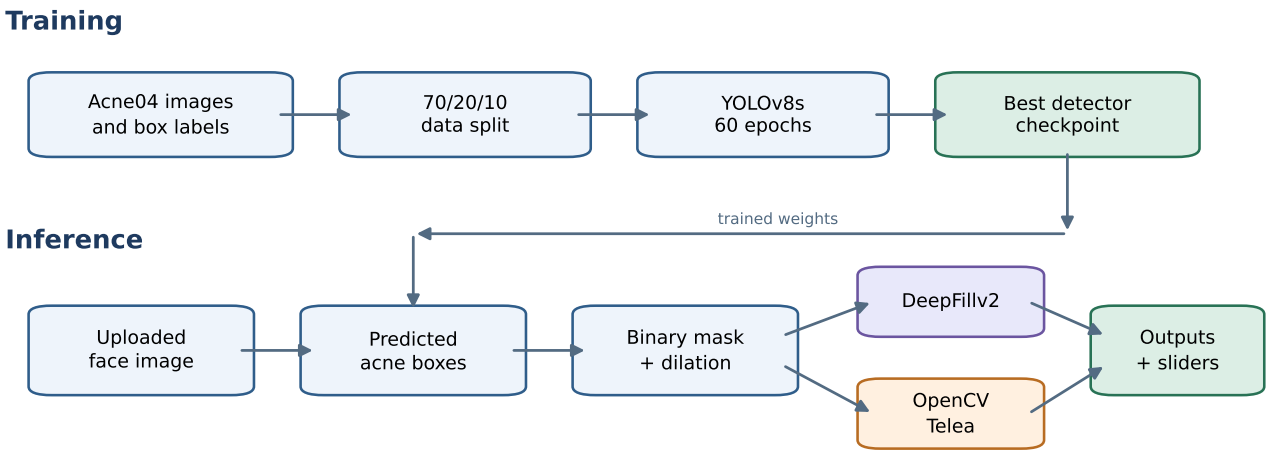


Figure 1. Overview of the proposed system. Acne04 box annotations train YOLOv8s. At inference time, the checkpoint localizes acne in an uploaded face, a dilated mask selects the edit regions, and DeepFillv2 and OpenCV Telea restore the same masked input.

a 70/20/10 split. The test images are withheld from parameter updates and checkpoint selection.

The detector starts from Ultralytics' pretrained yolov8s.pt weights and is trained for 60 epochs at 640-pixel input size, batch size 16, and AdamW optimization. The implemented augmentations include rotation up to 10 degrees, translation 0.1, scale 0.5, horizontal flip probability 0.5, mosaic 1.0, mixup 0.1, random erasing 0.4, and hue/saturation/value adjustments of 0.015/0.7/0.4. Vertical flip is disabled because an upside-down face is unlikely in the intended use. The run saves the best checkpoint as models/acne04_60_best.pt; the app loads it by default.

2.2. Detection and mask construction

For an input image I with height H and width W, YOLOv8s predicts a set of boxes $B_\tau = \{b_i\}$ after a confidence threshold τ. Each accepted box is filled into an initially zero-valued image. The source-resolution mask is

$$M_0(x,y) = \begin{cases} 1, & (x,y) \in \bigcup_{b_i \in B_\tau} b_i, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The app then applies a square morphological dilation kernel K for r iterations, giving $M = \text{dilate}_K^r(M_0)$. Dilation allows the fill to cover lesion edges just outside the predicted rectangle. The default controls are $\tau = 0.10$, a 9×9 kernel, and $r = 3$. These parameters are adjustable in the interface. A box is a coarse localization, so dilation can also include unaffected skin; the threshold and mask size therefore have a visible effect on the outcome.

2.3. Two inpainting paths

Both methods use the same input image and mask resized to 512×512 pixels. DeepFillv2 uses a pretrained gated-convolution generator [9]. The application downloads its inference code and weights on first launch, writes the image and mask to the expected input paths, runs the upstream inference script, and reads its output. This project did not retrain DeepFillv2.

The comparison path calls cv2.inpaint(image, mask, 3, cv2.INPAINT_TELEA), where 3 is the inpainting radius in pixels. Telea inpainting propagates nearby image information into masked regions [6]; it has no learned weights or training epochs. Keeping I and M fixed makes the two outputs directly comparable for this input, although the methods may behave differently on large or irregular masks.

2.4. User interaction

The Gradio app accepts an RGB face image and exposes controls for confidence, dilation kernel size, and dilation iterations. It shows the DeepFillv2 and OpenCV outputs separately. One draggable slider compares the original with DeepFillv2; a second compares OpenCV with DeepFillv2. If no acne boxes pass the confidence threshold, the app returns the original image rather than forcing an inpainting result.

3. Experimental Protocol

The experiments comprise a 60-epoch detector training run, validation metrics recorded at each epoch, a separate evaluation of the saved best checkpoint on the test

Setting	Value
Dataset	Acne04 YOLOv8, 1,457 pairs
Train / validation / test	1,019 / 292 / 146 images
Detector	YOLOv8s, pretrained initialization
Training	60 epochs, AdamW, batch 16
Detector input	640 pixels
Inpainting output	512×512 pixels
Default mask	$\tau = 0.10$, 9×9, 3 dilations
OpenCV radius	3 pixels, Telea method

Table 1. Dataset and implementation settings.

split, and qualitative inpainting comparisons. Table 1 records the principal settings. The complete per-epoch CSV and best checkpoint are available in the project repository [5].

Precision and recall evaluate predicted acne boxes. Average precision integrates the precision–recall relation at a chosen intersection-over-union (IoU) match threshold. We report mAP@50 at IoU 0.50 and mAP@50:95 averaged over IoU thresholds 0.50 through 0.95. The training file does not provide a separate mAP@95 value. These metrics quantify detection, not visual restoration quality.

4. Results and Analysis

4.1. Validation over 60 epochs

Figure 2 plots all 60 recorded validation points. Table 2 gives representative epochs. Validation mAP@50 rose from 0.01% at epoch 1 to 32.53% at epoch 57, with dips and recoveries rather than a monotonic increase. Epoch 57 also had the highest recorded mAP@50:95, 8.00%. At epoch 60, precision was slightly higher (40.43%), while recall, mAP@50, and mAP@50:95 were slightly lower than at epoch 57. This supports using the saved best checkpoint rather than assuming the final epoch is best. The gap between mAP@50 and mAP@50:95 suggests that accurate, tight localization remains difficult for small lesions.

4.2. Held-out detector evaluation

The saved checkpoint was evaluated once on the 146-image test split, which contains 1,776 labeled lesion instances. Table 3 compares this held-out result with the recorded epoch-57 validation result. Test mAP@50 is 32.79%, close to the validation value of 32.53%; the other test metrics are also slightly higher. This is evidence for the detector on this particular held-out split, not a guarantee for different lighting, cameras, skin tones, or acne types. No repeated random splits or confidence intervals were computed.

Epoch	P	R	mAP_{50}	$mAP_{50:95}$
1	0.03	0.67	0.01	0.00
10	17.01	28.29	11.08	2.76
20	31.36	32.27	21.97	5.20
30	32.59	34.95	24.38	5.82
40	29.00	33.40	21.06	5.20
50	32.91	38.69	25.20	6.23
57	40.37	42.20	32.53	8.00
60	40.43	40.19	31.77	7.70

Table 2. Validation detection metrics in percent. P is precision and R is recall; epoch 57 is the saved best checkpoint.

Split	P	R	mAP_{50}	$mAP_{50:95}$
Validation, epoch 57	40.37	42.20	32.53	8.00
Test, best checkpoint	42.39	42.74	32.79	8.51

Table 3. Detection metrics in percent on the validation and held-out test splits. The test row is from a separate evaluation of models/acne04_60_best.pt.

4.3. Mask and restoration comparison

Figure 3 shows an actual result for levle1_203.jpg, an image in the test split. At the default confidence and dilation settings, the detector produced 26 boxes and the resulting source-resolution mask covered 3.93% of the image. The white mask regions are exactly the areas targeted for filling. DeepFillv2 reduced several visible lesions while keeping more continuous nearby skin in this example. The OpenCV result removed content inside the same mask but shows conspicuous smooth patches around some rectangles. The observation is qualitative and applies to the displayed example; it is not a measured superiority claim across the dataset.

As a small mask-sensitivity diagnostic, the test image levle1_0.jpg was processed with two confidence thresholds while keeping the 9×9 kernel and three dilation iterations fixed. Table 4 shows that lowering the threshold from 0.10 to 0.05 increased the accepted boxes from 62 to 158 and the masked area from 18.82% to 27.39%. This illustrates why the interface exposes the threshold: an overly permissive detector can remove much more skin than intended. The diagnostic uses one image, so it does not estimate average threshold sensitivity.

5. Limitations and Conclusion

The detector uses boxes, not pixel-level lesion segmentation. Boxes and their dilation may mask normal skin, and missed detections leave lesions untouched. Resiz-

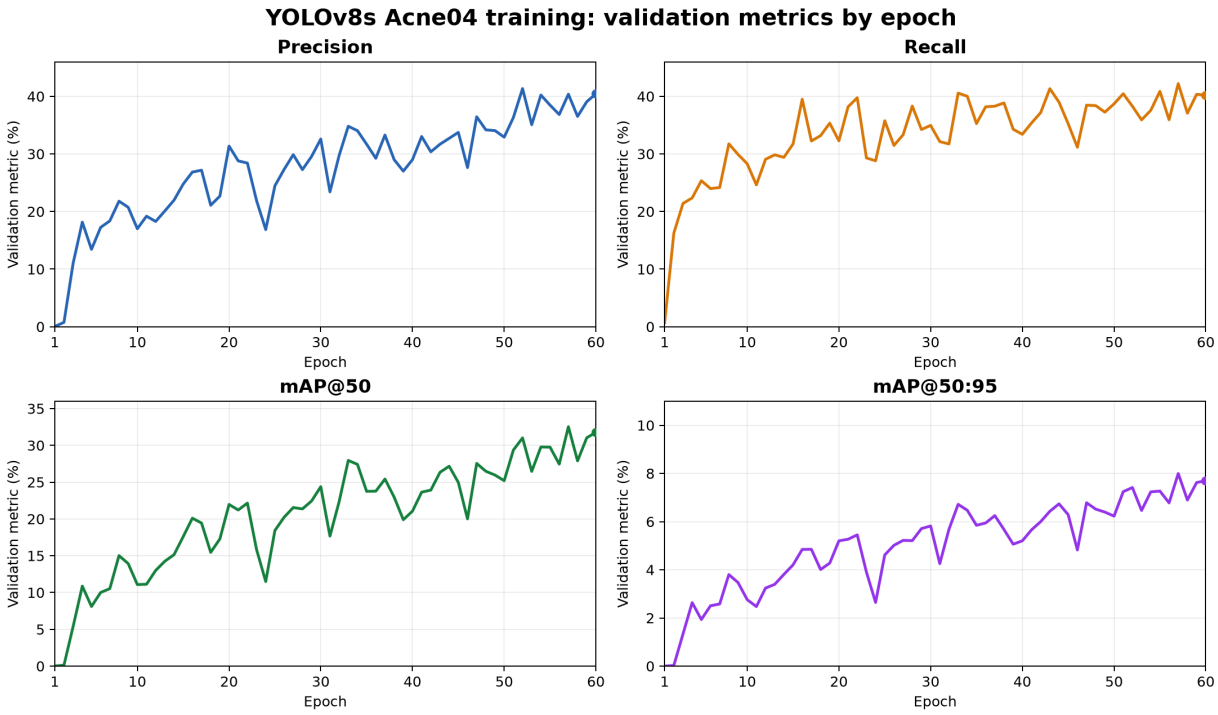


Figure 2. Validation precision, recall, mAP@50, and mAP@50:95 for each of the 60 YOLOv8s training epochs. Values are taken directly from the recorded results.csv.



Figure 3. Held-out Acne04 test image, predicted and dilated acne mask, DeepFillv2 result, and OpenCV Telea result. Settings: confidence 0.10, 9×9 dilation kernel, three dilation iterations, and Telea radius 3 pixels.

Confidence	Accepted boxes	Masked area (%)
0.05	158	27.39
0.10	62	18.82

Table 4. Mask sensitivity on one held-out image, levle1_0.jpg. Other mask settings were fixed.

ing every input to 512×512 for inpainting can distort non-square portraits. The example in Figure 3 exposes a limitation of the classical baseline on these rectangular masks, but the dataset has no paired clean target images. Consequently, PSNR, SSIM, or a defensible dataset-wide ranking of inpainting quality cannot be reported from these data. Neither the DeepFillv2 generator nor OpenCV Telea was trained on Acne04 in this project; the 60-epoch result applies to YOLOv8s only. The tool is for visual editing, not medical diagnosis.

This project delivers a working and inspectable acne-removal pipeline: an Acne04-trained detector creates a mask, two established inpainting methods fill the same regions, and sliders let users inspect their results. The held-out detector mAP@50 is 32.79%.

Future work should use pixel-level masks, assess performance across more varied portraits, and collect paired or human-rated restoration targets before making stronger claims about the comparative visual quality of the two methods.

6. Datasets and Source Repositories

The following links identify every dataset and source repository used directly in this project or its report:

- Project code and trained checkpoint: Pimple Removal Tool on GitHub [5].
- Training images and YOLO box labels: Acne04 YOLOv8 on Kaggle [1].
- Pipeline reference: EE604 Course Project [2].
- Pretrained inpainting implementation and weights: deepfillv2_colab [8].
- Detector framework: Ultralytics repository [7].
- Classical inpainting library: OpenCV repository [4].
- Other project dependencies: Gradio, PyTorch, NumPy, scikit-learn, gdown, tqdm, and scikit-image.
- Report figures: Matplotlib.
- Report format: CVPR author kit [3].

References

[1] Acne04 YOLOv8 dataset. Acne04 yolov8. Kaggle, https://www.kaggle.com/datasets/shreyash1110/acne04-yolov8, n.d. Accessed November 2026. 1, 5

[2] architk22. Ee604 course project. https://github.com/architk22/EE604-Course-Project, n.d. YOLOv8–DeepFillv2 pipeline reference. 1, 5

[3] CVPR. Cvpr author kit. https://github.com/cvpr-org/author-kit, n.d. CVPR 2026 LaTeX release. 5

[4] OpenCV. Opencv image inpainting. https://docs.opencv.org/4.x/df/d3d/tutorial_py_inpainting.html, n.d. Source repository: https://github.com/opencv/opencv. 1, 5

[5] Yogdeep Singh Rajawat. Pimple removal tool. https://github.com/blackleg-sanji007/pimple-removal-tool/tree/main, 2026. 3, 5

[6] Alexandru Telea. An image inpainting technique based on the fast marching method. Journal of Graphics Tools, 9(1):25–36, 2004. 1, 2

[7] Ultralytics. Ultralytics yolov8. https://github.com/ultralytics/ultralytics, n.d. Detector framework and training implementation. 1, 5

[8] vrindaprabhu. deepfillv2_colab. https://github.com/vrindaprabhu/deepfillv2_colab, n.d. Pretrained DeepFillv2 inference implementation and weights. 1, 5

[9] Jiahui Yu, Zhe Lin, Jimei Yang, Xiaohui Shen, Xin Lu, and Thomas S. Huang. Free-form image inpainting with gated convolution. In Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019. 1, 2

A. Complete Project-Authored Code

The following listings include the complete Python application and training script in the project repository, plus their dependency file. The binary detector checkpoint is available through the project repository. The externally maintained DeepFillv2 implementation is downloaded by the app and is linked in Section 6; it is not project-authored code. The listings are frozen copies for this report.

A.1. Application: app.py

```
"""Acne removal demo: YOLOv8s masks with DeepFillv2 and OpenCV inpainting."""

import os
import subprocess
import sys
from functools import lru_cache
from pathlib import Path

import cv2
import gdown
import gradio as gr
import numpy as np
import torch
from ultralytics import YOLO


ROOT = Path(__file__).resolve().parent
MODEL_PATH = Path(
    os.environ.get("ACNE_YOLO_MODEL_PATH", ROOT / "models" / "acne04_60_best.pt")
).expanduser()
if not MODEL_PATH.is_absolute():
    MODEL_PATH = ROOT / MODEL_PATH

DEEPFILL_DIR = ROOT / "deepfillv2_colab"
DEEPFILL_WEIGHTS = DEEPFILL_DIR / "model" / "deepfillv2_WGAN.pth"
DEEPFILL_REPO = "https://github.com/vrindaprabhu/deepfillv2_colab.git"
DEEPFILL_FILE_ID = "1uMghKl883-9hDLhSiI8lRbHCzCmmRwV-"
IMAGE_SIZE = (512, 512)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"


@lru_cache(maxsize=1)
def detector():
    if not MODEL_PATH.is_file():
        raise FileNotFoundError(f"YOLO checkpoint not found: {MODEL_PATH}")
    return YOLO(str(MODEL_PATH)).to(DEVICE)


@lru_cache(maxsize=1)
def ensure_deepfill():
    """Fetch the upstream inference code and its pretrained generator once."""
    inpaint_script = DEEPFILL_DIR / "inpaint.py"
    if not inpaint_script.is_file():
        if DEEPFILL_DIR.exists():
            raise RuntimeError(f"Incomplete DeepFill directory: {DEEPFILL_DIR}")
        subprocess.run(["git", "clone", DEEPFILL_REPO, str(DEEPFILL_DIR)],
                       check=True, cwd=ROOT)

    # The upstream script hardcodes eight workers for a one-image dataset.
    # Its config already sets NUM_WORKERS=0, which avoids Windows spawn delays.
    source = inpaint_script.read_text(encoding="utf-8")
    if "num_workers=8," in source:
        inpaint_script.write_text(
            source.replace("num_workers=8,", "num_workers=NUM_WORKERS,"),
            encoding="utf-8",
        )

```

```
    if not DEEPFILL_WEIGHTS.is_file():
        DEEPFILL_WEIGHTS.parent.mkdir(parents=True, exist_ok=True)
        result = gdown.download(id=DEEPFILL_FILE_ID,
                                output=str(DEEPFILL_WEIGHTS), quiet=False)
        if not result or not DEEPFILL_WEIGHTS.is_file():
            raise RuntimeError("DeepFill checkpoint download failed")


def acne_mask(image_bgr, confidence, kernel_size, iterations):
    height, width = image_bgr.shape[:2]
    boxes = detector().predict(
        image_bgr, conf=float(confidence), device=DEVICE, verbose=False
    )[0].boxes.xyxy
    mask = np.zeros((height, width), dtype=np.uint8)
    for box in boxes:
        x1, y1, x2, y2 = (int(value) for value in box.tolist())
        cv2.rectangle(mask, (x1, y1), (x2, y2), 255, -1)
    if len(boxes):
        size = max(3, int(kernel_size) | 1)
        kernel = np.ones((size, size), dtype=np.uint8)
        mask = cv2.dilate(mask, kernel, iterations=int(iterations))
    return mask, len(boxes)


def run_deepfill(image_bgr, mask):
    ensure_deepfill()
    image_path = DEEPFILL_DIR / "input" / "input_img.png"
    mask_path = DEEPFILL_DIR / "input" / "mask.png"
    output_path = DEEPFILL_DIR / "output" / "inpainted_img.png"
    image_path.parent.mkdir(parents=True, exist_ok=True)
    mask_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.parent.mkdir(parents=True, exist_ok=True)

    if not cv2.imwrite(str(image_path), image_bgr):
        raise RuntimeError("Could not save the DeepFill input image")
    if not cv2.imwrite(str(mask_path), mask):
        raise RuntimeError("Could not save the DeepFill mask")
    output_path.unlink(missing_ok=True)
    try:
        subprocess.run([sys.executable, "inpaint.py"], cwd=DEEPFILL_DIR,
                       check=True)
        result_bgr = cv2.imread(str(output_path))
        if result_bgr is None:
            raise RuntimeError("DeepFill produced no output image")
        return cv2.cvtColor(result_bgr, cv2.COLOR_BGR2RGB)
    finally:
        image_path.unlink(missing_ok=True)
        mask_path.unlink(missing_ok=True)
        output_path.unlink(missing_ok=True)


def compare_methods(image_rgb, confidence, kernel_size, iterations):
    if image_rgb is None:
        raise gr.Error("Upload a face image first.")
    image_rgb = np.asarray(image_rgb)
    image_bgr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)
    mask, detections = acne_mask(image_bgr, confidence, kernel_size, iterations)
    if detections == 0:
        gr.Info("No acne regions detected; showing the original image.")
        return image_rgb, image_rgb, (image_rgb, image_rgb), (image_rgb, image_rgb)

    resized_original = cv2.resize(image_rgb, IMAGE_SIZE)
    resized_bgr = cv2.resize(image_bgr, IMAGE_SIZE)
    resized_mask = cv2.resize(mask, IMAGE_SIZE)
    opencv_bgr = cv2.inpaint(resized_bgr, resized_mask, 3, cv2.INPAINT_TELEA)
    opencv_rgb = cv2.cvtColor(opencv_bgr, cv2.COLOR_BGR2RGB)
    deepfill_rgb = run_deepfill(image_bgr, mask)
```

```
    return (
        deepfill_rgb,
        opencv_rgb,
        (resized_original, deepfill_rgb),
        (opencv_rgb, deepfill_rgb),
    )


def create_gradio_app():
    with gr.Blocks() as demo:
        gr.Markdown("# Pimple Removal Tool (YOLOv8s + DeepFillv2)")
        gr.Markdown("Upload a face image and click **Remove Pimples**.")
        with gr.Row():
            with gr.Column():
                image_input = gr.Image(
                    type="numpy", image_mode="RGB", label="Input Face Image"
                )
                confidence = gr.Slider(
                    0.05, 0.5, value=0.1, step=0.01, label="Confidence Threshold"
                )
                kernel_size = gr.Slider(
                    3, 15, value=9, step=2, label="Mask Dilation Kernel Size (px)"
                )
                iterations = gr.Slider(
                    1, 5, value=3, step=1, label="Mask Dilation Iterations"
                )
                submit = gr.Button("Remove Pimples", variant="primary")
            with gr.Column():
                deepfill_output = gr.Image(type="numpy", label="DeepFillv2 Output")
                opencv_output = gr.Image(type="numpy", label="OpenCV Telea Output")
                before_after = gr.ImageSlider(label="Before / After DeepFillv2")
                methods = gr.ImageSlider(label="OpenCV / DeepFillv2")

        submit.click(
            fn=compare_methods,
            inputs=[image_input, confidence, kernel_size, iterations],
            outputs=[deepfill_output, opencv_output, before_after, methods],
            concurrency_limit=1,
        )
    return demo


if __name__ == "__main__":
    if not MODEL_PATH.is_file():
        raise SystemExit(f"YOLO checkpoint not found: {MODEL_PATH}")
    ensure_deepfill()
    share = os.environ.get("GRADIO_SHARE", "0").lower() in {"1", "true", "yes"}
    create_gradio_app().launch(
        server_name="127.0.0.1", share=share, theme=gr.themes.Soft()
    )
```

A.2. Detector training: train_acne04.py

```
"""Train a YOLOv8s acne detector on Acne04 for 60 epochs by default.

Expected raw dataset layout:
  data/acne04-yolov8/Images/*.jpg
  data/acne04-yolov8/labels/content/labels/*.txt
"""

import argparse
import os
import shutil
from pathlib import Path

from sklearn.model_selection import train_test_split
```

```
from ultralytics import YOLO
import torch


ROOT = Path(__file__).resolve().parent
IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png"}


def prepare_dataset(source: Path, output: Path) -> Path:
    image_dir = source / "Images"
    label_dir = source / "labels" / "content" / "labels"
    if not image_dir.is_dir() or not label_dir.is_dir():
        raise FileNotFoundError(
            f"Expected {image_dir} and {label_dir}. "
            "Place the Acne04 YOLOv8 download there or pass --source."
        )

    pairs = []
    for image in sorted(image_dir.iterdir()):
        if image.suffix.lower() not in IMAGE_EXTENSIONS:
            continue
        label = label_dir / f"{image.stem}.txt"
        if label.is_file():
            pairs.append((image, label))
    if len(pairs) < 10:
        raise ValueError(f"Only {len(pairs)} image/label pairs found in {source}")

    train_val, test = train_test_split(pairs, test_size=0.1, random_state=42)
    train, val = train_test_split(train_val, test_size=2 / 9, random_state=42)
    for name, split in (("train", train), ("val", val), ("test", test)):
        images_out = output / "images" / name
        labels_out = output / "labels" / name
        images_out.mkdir(parents=True, exist_ok=True)
        labels_out.mkdir(parents=True, exist_ok=True)
        for image, label in split:
            shutil.copy2(image, images_out / image.name)
            shutil.copy2(label, labels_out / label.name)

    yaml_path = output / "data.yaml"
    yaml_path.write_text(
        f"path: {output.resolve().as_posix()}\n"
        "train: images/train\nval: images/val\ntest: images/test\n"
        "names:\n 0: pimple\n",
        encoding="utf-8",
    )
    print(f"Prepared Acne04: {len(train)} train, {len(val)} val, {len(test)} test")
    return yaml_path


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--source", type=Path, default=ROOT / "data" / "acne04-yolov8")
    parser.add_argument("--output", type=Path, default=ROOT / "data" / "acne_dataset")
    parser.add_argument("--epochs", type=int, default=60)
    parser.add_argument("--batch", type=int, default=16)
    parser.add_argument("--workers", type=int, default=2 if os.name == "nt" else 8)
    parser.add_argument("--resume", action="store_true",
                        help="Resume the current 60 epoch run from last.pt")
    args = parser.parse_args()

    if args.resume:
        checkpoint = ROOT / "runs" / "yolov8s_acne_augmented_60" / "weights" / "last.pt"
        if not checkpoint.is_file():
            raise FileNotFoundError(f"No training checkpoint at {checkpoint}")
        YOLO(str(checkpoint)).train(resume=True)
        return

```

```
    data_yaml = prepare_dataset(args.source, args.output)
    device = 0 if torch.cuda.is_available() else "cpu"
    model = YOLO("yolov8s.pt")
    model.train(
        data=str(data_yaml),
        epochs=args.epochs,
        imgsz=640,
        batch=args.batch,
        name="yolov8s_acne_augmented_60",
        project=str(ROOT / "runs"),
        device=device,
        optimizer="AdamW",
        patience=args.epochs,
        workers=args.workers,
        exist_ok=True,
        verbose=True,
        degrees=10.0,
        translate=0.1,
        scale=0.5,
        shear=0.0,
        perspective=0.0,
        flipud=0.0,
        fliplr=0.5,
        mosaic=1.0,
        mixup=0.1,
        erasing=0.4,
        hsv_h=0.015,
        hsv_s=0.7,
        hsv_v=0.4,
    )


if __name__ == "__main__":
    main()
```

A.3. Dependencies: requirements.txt

```
gradio>=6,<7
ultralytics
gdown
tqdm
opencv-python-headless
scikit-image
scikit-learn
```